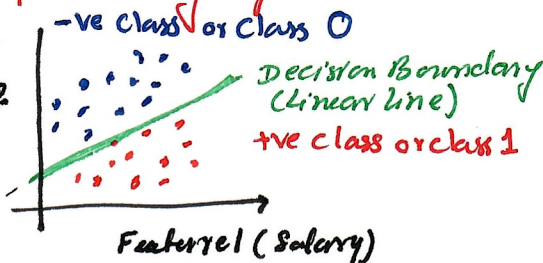


Linear Perceptron:

- A **binary classifier** that makes decisions by computing a **weighted sum of input** features and applying a **sign function**: $\hat{y} = \text{sign}(w^T x + b)$ $\rightarrow$ (or fits). It separates data using a **linear decision boundary**.

Acronyms PLA - Perceptron Learning Algorithm

Perceptron Ex (Debt)



Task - Credit Approval
 +ve class (class 1) $\rightarrow$ Approve
 -ve class (class 0) $\rightarrow$ Disapprove

Input Vector $\vec{x} = [x_1, \dots, x_d]^T$, Weight (Feature Importance) vector $\vec{w} = [w_1, \dots, w_d]$

Credit Score = $\vec{x}^T \vec{w} = \sum_{i=1}^d w_i x_i$

Approve Credit if $\sum_{i=1}^d w_i x_i \geq \text{Threshold}$; Reject credit if $\sum_{i=1}^d w_i x_i < \text{Threshold}$

Since it is a binary classification,

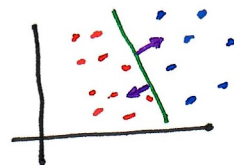
define $h(x) = \text{Sign}\left(\sum_{i=1}^d w_i x_i + w_0\right) = \text{Sign}(\vec{w}^T \vec{x})$, where
 $\vec{w} = [w_0, w_1, \dots, w_d]^T$, $\vec{x} = [1, x_1, \dots, x_d]^T$

Formal Representation of Perceptron

Input: $\vec{x} \in \mathbb{R}^d$ (Features or attributes)

Output: $y \in \{-1, +1\}$ (Labels)

Model: $h(\vec{x}) = \text{Sign}(\vec{w}^T \vec{x}) = \begin{cases} +1 & \text{if } \vec{w}^T \vec{x} > 0 \\ -1 & \text{if } \vec{w}^T \vec{x} < 0 \end{cases}$



Problem: How to find a hyperplane that separates the data?

First note that $\vec{w}^T \vec{x} = [w_0, w_1, w_2, \dots, w_d] \cdot [1, x_1, x_2, \dots, x_d]$ from now on.

w_0 corresponds to the Bias Term (In 2D it is the intercept on Y-axis)

$\vec{w}^T \vec{x} = \underbrace{w_0}_{\text{Bias}} + w_1 x_1 + \dots + w_n x_n$ (Called Inner Product of $\vec{w}$ & $\vec{x}$)

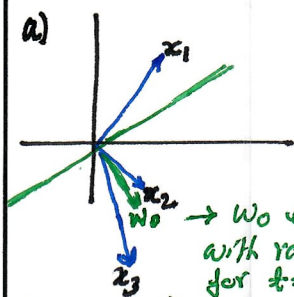
Our Approach: Start with some vector $\vec{w}$ & then adjust adjust the values of $w_0, w_1, \dots, w_d$ so that we can separate data into two classes.

$w_0 + w_1 x_1 + \dots + w_d x_d$ is called INNER PRODUCT of $\vec{w}$ & $\vec{x}$

A hyperplane: Is a decision boundary defined by $\vec{w}^T \vec{x} + b = 0$. called hyperplane because it is a flat, $(n-1)$ dimensional surface in n -dim space

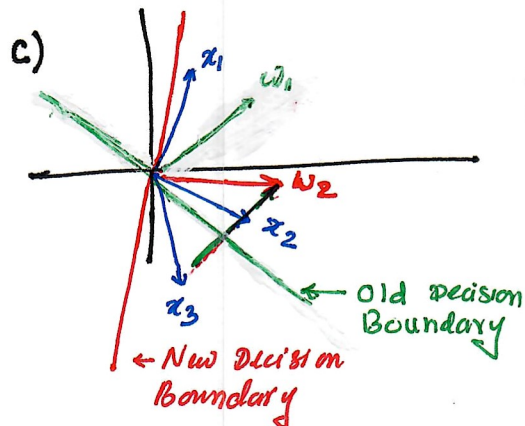
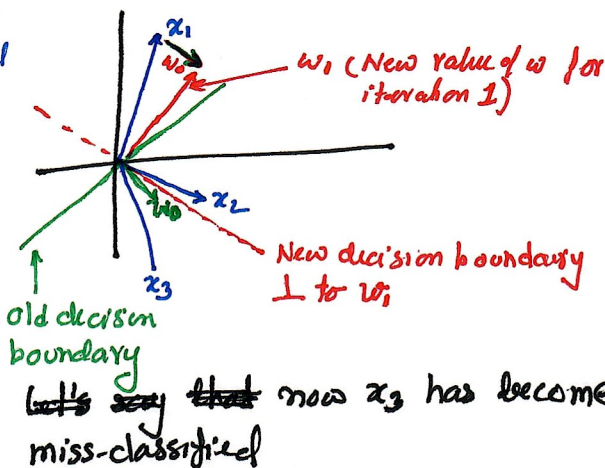
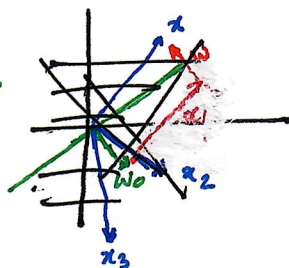
PLA - Perceptron Learning Algorithm

- Start with a random initialization of $\vec{w}$, then update $\vec{w}$'s iteratively by addressing incorrectly classified samples. For the following ex, assume that all x_1, x_2 & x_3 belong to the same class



Random $\vec{w}$. ~~Let~~
~~say~~ x_1 is incorrectly classified

b) update the hyperplane by adding x_1 to w_0 . Call the new value w_1



Now all x_1, x_2 & x_3 are classified correctly.

The red decision boundary in this figure now represents the optimum $\vec{w}$

$\vec{w}$ has been fine tuned. Note that successive corrections get smaller & smaller

Algorithm

Iterative Incremental Learning on single example at a time

- 1) Initialize $\vec{w}(0) = \vec{0}$ (Or some other random real initial value)
- 2) for step $t = 1, 2, 3, \dots$
 - a) Pick one miss-classified sample $(\vec{x}_s, y_s)$
 - We know (Because it is miss-classified that
 - if $y_s = 1$, then $\vec{w}(t)^T \vec{x}_s = -1$ & if $y_s = -1$, then $\vec{w}(t)^T \vec{x}_s = +1$
 - c.i.e. $\text{sign}(\vec{w}(t)^T \vec{x}_s) \neq y_s$

b) Update the weight

$$\vec{w}(t+1) = \vec{w}(t) + y_s \vec{x}_s$$

c) $t \leftarrow t+1$

Note:

- 1) We can choose to stop when there is no missclassified sample anymore
- 2) Or we can stop when Δw is smaller than some threshold ϵ

caveat: If our sample are not linearly separable, then they CAN NOT be ALL correctly classified by LP

PLA ContinuedAlgorithm Learning Rate (LR)

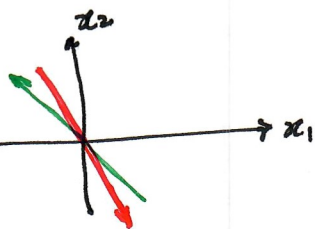
- a) $\vec{w}$ are not necessarily normalized
- If $\vec{w}(t) \gg \vec{x}_s$, then $\vec{w}(t) + \vec{x}_s \approx \vec{w}(t)$
 - Or if x is an outlier, the decision boundary may get redefined drastically.
- b) Solution use LR $\alpha > 0$
- $\vec{w}(t+1) = \vec{w}(t) + \alpha \vec{x}_s$
have big
 - If $\alpha \gg 0$, $\vec{x}_s$ will ^{have big} affect $\vec{w}(t+1)$ largely
 - If $\alpha \approx 0$, $\vec{x}_s$ would have little impact on $\vec{w}(t+1)$
 - LR is a hyperparameter. Can not be LEARNED from data BUT depends on the problem data.
 - LR can start big & decrease as time goes on i.e. $\alpha = \frac{1}{t} \cdot \text{Some Const}$

PLA Convergence

- Considering 1 sample at a time may destroy classification of other samples.
- PLA is guaranteed to reach correct soln in finite steps IF the data can be fit by a linear separator (Linearly separable)
- If the data can not be fit by a perceptron, infinitely many combinations of Perceptron can fit it $\rightarrow$ NEURAL NETWORKS.

PLA Cost

- PLA is Local, Greedy algorithm & can lead to exponential number of ^{updates} w
- Worst Case - Almost anti-parallel vectors that must be classified in the same class (or half space)



- PLA will rotate separation line in one of the 2 directions
- PLA will require more & more time as the angle between two vectors approaches 180°

Parametric v/s Non-parametric Models

Parametric Models - Faster, Strong assumption about data (ex Linear Perceptrons, Linear Regression)
- Depend on params that can be learned from data (In our case w was that param)

Non-Parametric Models - More Flexible, No assumptions abt data, (Ex KNN)
(Instance/memory based) - Computationally intractable for large dataset

Activation Function

Multi-Layer Perceptron (MLP)

- Several Perceptrons stacked in layer.

SLP

- In Perceptron (or Single Layer Perceptron SLP) $h(x) = \text{sign}(\vec{w}^T x)$, the sign function is the Activation Function.

- $\vec{w}^T x$ is called Pre-Activations

MLP

- A hidden unit at layer l is: $z_l = \phi_l(b_l + W_l z_{l-1})$, where ϕ_l - Activation Function & $b_l + W_l z_{l-1}$ is Pre Activation

We can choose the activation functions as we please but if all activation functions are linear: $\phi_l(a) = C_l(a)$

- Note that $f(z; \theta) = W_l C_l(W_{l-1} C_{l-1}(\dots W_1 x \dots)) = Wx$ Will be Linear.
this ex. did not include bias BUT the concept holds even if we add bias

Summary: - In Perceptron, an activation function is applied to pre-activations

- Non-Linearity in activation function is a reqd to capture non-linear relationship in the data.
- If all activation functions are Linear then the whole MLP collapses to a linear function.

MLP (Multi-layer Perceptrons)

SLP have limits - Cannot capture non-linear relationships

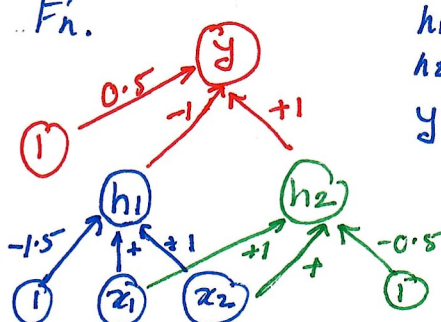
- Decision boundaries are linear
- XOR Problem

XOR Problems

x_1	x_2	$x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0
$(0,1) = 1$		$(1,1) = 0$
$(0,0) = 0$		$(1,0) = 1$

Not Linearly Separable. Hence can not be represented with Perceptron

XOR problem requires a MLP with a hidden layer & a non-linear Activation F_n.



$$h_1 = x_1 \wedge x_2 \quad (x_1 \& x_2) \quad [\text{AND}]$$

$$h_2 = x_1 \vee x_2 \quad (x_1 | x_2) \quad [\text{OR}]$$

$$y = \neg h_1 \wedge h_2 \quad ((!h_1) \& h_2)$$

h_1 & h_2 are hidden units

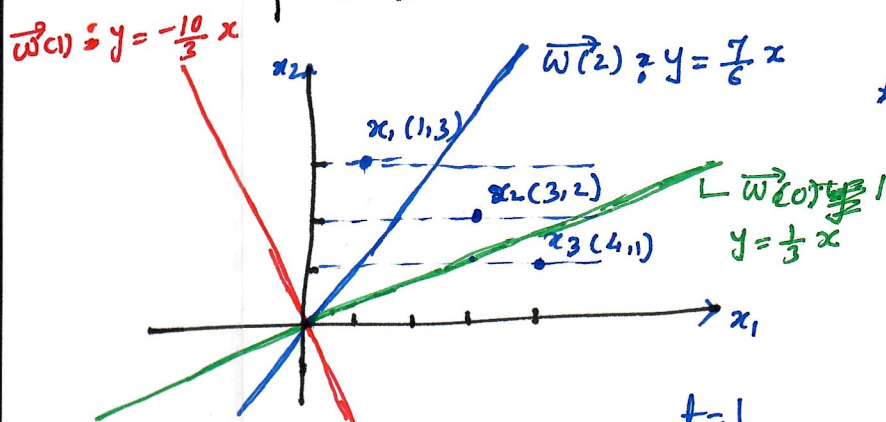
Please see the book

MLP maps input to higher Dimensional Space, where XOR pattern becomes Linearly Separable
MLPs can represent ANY Logical Function

Linear Perceptron Example

Sample	Features (Coord)	True Class
x_1	$[1, 3]^T$	1 ($y_1 = 1$)
x_2	$[3, 2]^T$	2 ($y_2 = -1$)
x_3	$[4, 1]^T$	2 ($y_3 = -1$)

- Linear Perceptron is initialized with random $\vec{w}$
 $\vec{w}(0) = [-\frac{1}{3}, 1]$
- For simplicity the intercept is assumed to be 0
 $\Rightarrow$ No need for w_0 & x_0



* $\vec{w}(0) = [-\frac{1}{3}, 1]$ means
 $-\frac{1}{3}x + y = 0 \Rightarrow y = \frac{1}{3}x$

Initial $t=0$

a) $\vec{w}(0) = [-\frac{1}{3}, 1]$ $\begin{bmatrix} 1 \\ 3 \end{bmatrix} \Rightarrow \vec{w}_0^T x_1 = 0$

$h(x_1) = \text{sign}((-\frac{1}{3}) \cdot (1) + (1) \cdot (3))$

$\hat{y}_1 = \text{sign}(8/3) = 1$

$\Rightarrow \hat{y}_1 = y_1 \Rightarrow x_1$ is correctly classified

b) $h(x_2) = [-\frac{1}{3}, 1] \begin{bmatrix} 3 \\ 2 \end{bmatrix}$

$h(x_2) = \text{sign}((-\frac{1}{3}) \cdot (3) + (1) \cdot (2))$

$= \hat{y}_2 = \text{sign}(1) = 1$

$\hat{y}_2 = 1, \hat{y}_2 = 1$

$y_2 x_2$ is incorrectly classified

c) $h(x_3) = \hat{y}_3 = 2 = y_3$
 x_3 is correctly classified

$t=1$

x_2 was incorrectly classified in $t=0$

$\vec{w}(0) = [-\frac{1}{3}, 1]$

$x_3 = [4, 1]$

$\vec{w}(1) = \vec{w}(0) + y_2 x_2$

$= [-\frac{1}{3}, 1] + (-1) [3, 2]^T$

$= [-\frac{1}{3}, 1] + [-3, -2]^T$

$= [-\frac{10}{3}, -1]^T$

$\Rightarrow \vec{w}(1) = [-\frac{10}{3}, -1]$

or $-\frac{10}{3}x - y = 0$

$\Rightarrow y = -\frac{10}{3}x$

$\hat{y}_1 = h(x_1) = \text{sign}([- \frac{10}{3}, -1] \begin{bmatrix} 1 \\ 3 \end{bmatrix})$

$\hat{y}_1 = -1 \neq y_1$ miss classified

$\hat{y}_2 = y_2$ good
 $\hat{y}_3 = y_3$ good

$t=2$

In $t=1$, x_1 was missclassified

$\vec{w}(1) = [-\frac{10}{3}, -1]^T$

$x_1 = [1, 3]^T$

$\vec{w}(2) = \vec{w}(1) + y_1 x_1$

$= [-\frac{10}{3}, -1]^T + (1) [1, 3]^T$

$= [-\frac{7}{3}, 2]^T$

$\vec{w}(2) = [-\frac{7}{3}, 2]^T$

$\Rightarrow y = \frac{7}{6}x$

Now all samples were correctly classified